



Project Report

Garage Service Management System

Only for course Teacher						
		Needs Improvement	Developing	Sufficient	Above Average	Total Mark
Allocate mark & Percentage		25%	50%	75%	100%	15
Understanding / Analysis	4					
Implementation	5					
Report Writing	6					
Total obtained mark						
Comments						

Semester: Spring 2025

Group No:- (6)

Name: Md.Kawsher Ahmed

ID: REDACTED

Name: Shafiur Rahman

ID: REDACTED

Name: Arnob Kumar Paul

ID: REDACTED

Batch: 43

Section: C2

Course Code: SE 133

Course Name: Software Development Capstone Project

Course Teacher Name: Abdul Hye Zebon

Designation: Lecturer

Submission Date: 19/08/2025

Garage Service Management System

Table of Contents

1. Introduction.....	02
1.1 System Overview.....	02
1.2 Why we developed this system?.....	02
2. Stakeholders.....	02
2.1 What is a Stakeholder?.....	02
2.2 Who are the Stakeholders for this system?.....	03
3. Requirement Analysis.....	03
3.1 What is Requirement Analysis?.....	03
3.2 Functional Requirements.....	03
3.3 Non-Functional Requirements.....	03
3.4 Customer and Vehicle Management.....	04
3.5 Service Management.....	05
3.6 BillingandInvoiceGeneration.....	06
3.7 Daily Revenue Reports.....	08
3.8 System Navigation.....	08
3.9 Non-Functional Requirements.....	10
3.10 Usability.....	11
3.11 Reliability.....	12
3.12 Performance.....	12
3.13 Portability.....	12
3.14 Security.....	13
3.15 Maintainability.....	13
3.16 Scalability.....	14
3.17 Compatibility.....	15
3.18 Constraints.....	15
4. Use Case Diagram.....	15
5. User Manual.....	18
5.1 Feature Description for Authentication.....	19
5.2 Feature Description for Customer & Vehicle Management.....	22
5.3 Feature Description for Service Management.....	23
5.4 Feature Description for Billing & Invoices.....	24
5.5 Feature Description for Reports.....	26
6. Project Plan.....	27
7. Testing Analysis for the whole system.....	29
8. Conclusion.....	31

1. Introduction

In today's fast-paced automotive service environment, accuracy, efficiency, and reliability are paramount. A garage requires precise customer and vehicle management to avoid mismanagement or delays and timely service to meet the demands of vehicle owners and mechanics. Manual management of these tasks can be prone to errors, time-consuming, and challenging, particularly in larger operations.

1.1 System Overview

The Garage Service Management System (GSMS) is a software platform designed to streamline the operations of garages by managing customers, vehicles, services, billing, and reports. Built with a modular and scalable architecture in C programming, it ensures user-friendly functionality. Suitable for small garages and large service centers, the system enhances efficiency, minimizes errors, and simplifies complex workflows. Here's a breakdown of the key components:

- Tracks customers and vehicles in real time.
- Facilitates fast and accurate service management.
- Generates detailed invoices and maintains transaction histories.
- Provides authentication-based access control to safeguard sensitive data.
- Generates daily revenue reports.

1.2 Why we developed this system?

The Garage Service Management System was developed to streamline garage operations by automating processes, reducing errors, and improving efficiency. It ensures accurate customer and vehicle tracking, enhances service management, and supports billing compliance. The system provides real-time analytics for data-driven decisions, facilitates scalability for growing businesses, and lowers operational costs by minimizing manual errors and inefficiencies.

2. Stakeholders

2.1 What is a Stakeholder?

Stakeholders in a software system are individuals, groups, or organizations that are impacted by the system's development, use, or existence. They have a vested interest in the system's success and can influence its requirements, design, and functionality.

2.2 Who are the Stakeholders for this system?

There is one primary stakeholder for this system: the Admin (Garage Manager). We've created a single interface with authentication for secure access. When the stakeholder uses this system, they can access all features after logging.

- **Non-logged Users:** Non-logged users cannot access any features of this system. One must log in to access the features.
- **Logged Admin:** Logged admin can access all features like managing customers and vehicles, adding/updating services, generating invoices, and viewing reports.

3. Requirement Analysis

3.1 What is Requirement Analysis?

Requirements Analysis is a fundamental stage in software development that focuses on determining the needs, expectations, and conditions that a new or modified product or system should fulfill. This process considers the often conflicting requirements of various stakeholders involved in the project.

3.2 Functional Requirements for this system

Functional requirements describe the specific features, behaviors, and capabilities that the GSMS must provide to meet the needs of a garage management operation. These requirements are derived from the system's core functionalities: user authentication, customer and vehicle management, service management, billing and invoice generation, daily revenue reporting, and system navigation.

3.3 User Authentication System

The system includes a basic authentication mechanism to ensure only authorized users can access its features.

❖ FR1.1: User Login

- **Description:** The system shall allow users to log in using a predefined username and password combination (default: "admin"/"admin").
- **Details:** The login function (login_user) prompts for a username and password, validates them against hardcoded credentials using validate_credentials, and sets a

global flag (`user_logged_in`) to true upon success. The system displays a welcome message with the username and pauses for user acknowledgment.

- **Constraints:** Only one set of credentials is supported, and no password encryption is implemented for simplicity.

❖ **FR1.2: Credential Validation**

- **Description:** The system shall validate user credentials and grant access only if they match the predefined values.
- **Details:** The `validate_credentials` function performs a case-sensitive string comparison using `strcmp`. Invalid credentials result in an error message, and the user is prompted to try again or exit.
- **Constraints:** Hardcoded credentials limit flexibility, and there is no support for multiple users or password recovery.

❖ **FR1.3: User Logout**

- **Description:** The system shall provide a logout function to end the current user session.
- **Details:** The `logout_user` function clears the `user_logged_in` flag and resets the `current_username` to an empty string. It displays a confirmation message and pauses for acknowledgment.
- **Constraints:** Logout is only available when a user is logged in, and the system checks the login status to prevent unnecessary logout attempts.

❖ **FR1.4: Access Restriction**

- **Description:** The system shall restrict access to all management functionalities (customer, service, billing, reports) until a user is authenticated.
- **Details:** The `display_main_menu` function shows a limited menu (login or exit) when `user_logged_in` is false. All other menu options are only accessible after successful login.
- **Constraints:** The system assumes a single-user environment with no role-based access control.

3.4 Customer and Vehicle Management

This module manages customer and vehicle information, integrating both into a single record for simplicity, as is common in small garage operations.

❖ **FR2.1: Add Customer and Vehicle**

- **Description:** The system shall allow users to add a new customer with associated vehicle details, including customer ID, name, phone, address, vehicle plate, make, model, year, and color.
- **Details:** The `add_customer` function prompts for customer details (name, phone, address) and vehicle details (license plate, make, model, year, color) using `fgets` for safe input. It generates a unique customer ID using `get_formatted_id` and saves the data to `data/customers.txt` in the format `ID|Name|Phone|Address|Plate|Make|Model|Year|Color`. A confirmation message displays the entered details.
- **Constraints:** Input fields are limited by predefined sizes (e.g., 50 characters for name, 20 for phone). No validation is performed on input formats (e.g., phone number or license plate).

❖ **FR2.2: Automatic Customer ID Generation**

- **Description:** The system shall automatically generate unique customer IDs in the format "001", "002", etc.
- **Details:** The `get_formatted_id` function with `ID_CUSTOMER` type uses a global counter (`customer_count`) initialized by checking the highest existing ID in `customers.txt` via `get_highest_customer_id`. IDs are formatted as three-digit strings (e.g., "001").
- **Constraints:** IDs are limited to three digits (up to 999 customers). Duplicate IDs are prevented by incrementing the counter.

❖ **FR2.3: Persistent Customer Storage**

- **Description:** The system shall store customer and vehicle data in a text file (`customers.txt`) for persistence.
- **Details:** Data is appended to `customers.txt` in a pipe-delimited format. The `create_data_directory` function ensures the data directory exists before writing.
- **Constraints:** File operations use append mode ("a"), and errors (e.g., disk full) result in a failure message without retry mechanisms.

❖ **FR2.4: Display All Customers**

- **Description:** The system shall display a list of all customers and their vehicles in a tabular format, showing ID, name, phone, plate, make, and model.
- **Details:** The `display_all_customers` function reads `customers.txt`, parses each line using `sscanf`, and prints a formatted table. It skips empty lines and ensures at least basic customer info (ID, name, phone, address) is present before displaying.
- **Constraints:** The table displays only a subset of fields for brevity. Missing or malformed data may be displayed as empty strings.

❖ **FR2.5: Search Customer by ID**

- **Description:** The system shall allow users to search for a customer by their ID and display their full details, including vehicle information.
- **Details:** The `search_customer_by_id` function reads `customers.txt`, matches the provided ID, and displays all customer and vehicle details if found. It returns 1 for success and 0 for failure, with an appropriate message.
- **Constraints:** The search is case-sensitive and requires exact ID matches. No partial search or fuzzy matching is supported.

3.5 Service Management

This module handles the creation, viewing, and updating of service records for garage operations.

❖ **FR3.1: Add New Service**

- **Description:** The system shall allow users to add new services with details including service ID, customer ID, service name, date, status, and cost.
- **Details:** The `addService` function prompts for customer ID, service name, and cost. It generates a service ID using `get_formatted_id`, sets the current date using `getCurrentDate`, and assigns an initial "Pending" status. The service is stored in an in-memory array (`services`) and written to `services.txt`.
- **Constraints:** Limited to 100 services (`MAX_SERVICES`). Cost must be a positive number, but no format validation is enforced for the service name.

❖ **FR3.2: Automatic Service ID Generation**

- **Description:** The system shall generate unique service IDs in the format "001", "002", etc.
- **Details:** The `get_formatted_id` function with `ID_SERVICE` type uses a global counter (`service_count_id`) to generate three-digit IDs. The counter increments with each new service.
- **Constraints:** Limited to 999 services due to the three-digit format. No rollback mechanism exists if file saving fails.

❖ **FR3.3: Customer ID Validation**

- **Description:** The system shall validate customer IDs before creating a service, with an option to proceed if the ID is not found.
- **Details:** The `customerExists` function checks if the customer ID exists using `search_customer_by_id`. If not found, the user is warned and can choose to continue or cancel the service creation.
- **Constraints:** Validation is optional, allowing services to be created for non-existent customers, which may lead to data inconsistencies.

❖ **FR3.4: Persistent Service Storage**

- **Description:** The system shall store service data in a text file (`services.txt`) in the format `ServiceID|CustomerID|ServiceName|Date|Status|Cost`.
- **Details:** The `addService` function overwrites `services.txt` with the entire services array after adding a new service. The file is opened in write mode ("w"), ensuring all services are saved.
- **Constraints:** Overwriting the file risks data loss if the write operation fails. No backup mechanism is implemented.

❖ **FR3.5: View All Services**

- **Description:** The system shall display all services in a tabular format, showing service ID, customer ID, service name, date, status, and cost.
- **Details:** The `viewAllServices` function prints the services array in a formatted table. If no services exist, a message is displayed.
- **Constraints:** Limited to in-memory services (up to 100). The table may become unwieldy for large datasets due to fixed-width formatting.

❖ **FR3.6: Update Service Status**

- **Description:** The system shall allow users to update the status of a service to "Pending", "In Progress", or "Completed" by entering the service ID.
- **Details:** The `updateServiceStatus` function prompts for a service ID, displays the current status, and allows the user to select a new status via a numbered menu. Changes are saved to `services.txt` by overwriting the file.
- **Constraints:** Only three predefined statuses are supported. Invalid service IDs result in an error message without retry options.

3.6 Billing and Invoice Generation

This module manages invoice creation and viewing for completed services.

❖ **FR4.1: Generate Invoice**

- **Description:** The system shall allow users to generate invoices for completed services, including invoice ID, service ID, customer ID, customer name, service name, invoice date, total amount, and payment status.
- **Details:** The generateSimpleInvoice function lists completed services from services.txt, allows the user to select one, retrieves the customer name from customers.txt, and creates an invoice with a unique ID ("INV001", etc.). The invoice is marked as "Paid" and saved to invoices.txt.
- **Constraints:** Limited to 50 invoices (MAX_INVOICES). Only completed services can be invoiced, and the total amount equals the service cost without additional taxes or fees.
- ❖ **FR4.2: Automatic Invoice ID Generation**
 - **Description:** The system shall generate unique invoice IDs in the format "INV001", "INV002", etc.
 - **Details:** The get_formatted_id function with ID_INVOICE type uses a global counter (invoice_count_id) to generate IDs. The counter increments with each new invoice.
 - **Constraints:** Limited to 999 invoices due to the three-digit format. No validation ensures service IDs are unique in invoices.
- ❖ **FR4.3: Customer Name Retrieval**
 - **Description:** The system shall retrieve customer names from customers.txt for invoice generation.
 - **Details:** The generateSimpleInvoice function parses customers.txt to find the customer name matching the service's customer ID. If not found, it defaults to "Unknown Customer".
 - **Constraints:** Assumes customer ID exists in customers.txt for accurate naming. Parsing errors may result in incorrect names.
- ❖ **FR4.4: Persistent Invoice Storage**
 - **Description:** The system shall store invoice data in a text file (invoices.txt) in the format
InvoiceID|ServiceID|CustomerID|CustomerName|ServiceName|InvoiceDate|Total Amount|PaymentStatus.
 - **Details:** The saveSimpleInvoices function overwrites invoices.txt with the entire invoices array after adding a new invoice. The loadSimpleInvoices function populates the array on startup.
 - **Constraints:** Overwriting risks data loss if the write fails. No validation ensures data consistency across files.
- ❖ **FR4.5: Display Completed Services for Invoicing**
 - **Description:** The system shall display a list of completed services for invoice generation, allowing users to select a service by number.
 - **Details:** The generateSimpleInvoice function filters services with "Completed" status from services.txt and displays them in a numbered table. Users select a service by entering its number.
 - **Constraints:** Limited to 50 temporary services in memory during invoice generation. Invalid selections result in an error without retry.
- ❖ **FR4.6: Print Invoice Receipt**

- **Description:** The system shall provide an option to print a formatted receipt for generated invoices, displaying invoice ID, date, customer details, service details, total amount, and payment status.
- **Details:** After invoice creation, `generateSimpleInvoice` prompts the user to print a receipt. If selected, it displays a formatted text receipt with a professional layout, including a "Thank you" message.
- **Constraints:** Receipts are printed to the console only, with no option for physical printing or file export.
- ❖ **FR4.7: View All Invoices**
 - **Description:** The system shall display all invoices in a tabular format, showing invoice ID, customer name, service name, amount, and payment status.
 - **Details:** The `viewAllInvoices` function prints the invoices array in a formatted table. If no invoices exist, a message is displayed.
 - **Constraints:** Limited to 50 invoices in memory. Table formatting may misalign for long names or services.

3.7 Daily Revenue Reports

This module provides financial reporting for daily operations.

- ❖ **FR5.1: Generate Daily Revenue Report**
 - **Description:** The system shall generate a report for the current date, listing all invoices issued that day with details including invoice ID, customer name, service name, amount, and payment status.
 - **Details:** The `dailyRevenueReport` function reads `invoices.txt`, filters invoices by the current date (obtained via `getCurrentReportDate`), and displays them in a table. It calculates the total revenue and invoice count.
 - **Constraints:** Only invoices from the current day are included. Date comparison is exact, with no support for custom date ranges.
- ❖ **FR5.2: Calculate Total Revenue**
 - **Description:** The system shall calculate and display the total number of invoices and total revenue for the current day.
 - **Details:** The `dailyRevenueReport` function sums the `total_amount` of today's invoices and counts them, displaying the results below the invoice table.
 - **Constraints:** Revenue is based on invoice amounts, assuming all are paid. No support for partial payments or refunds.

3.8 System Navigation

The system provides a menu-driven interface for user interaction.

- ❖ **FR6.1: Main Menu for Authenticated Users**
 - **Description:** The system shall provide a main menu with options for customer management, service management, billing, reports, logout, and exit when a user is logged in.

- **Details:** The `display_main_menu` function shows a formatted menu with options 1–5 and 0 when `user_logged_in` is true. Each option leads to a corresponding module or action.
- **Constraints:** Menu options are fixed and cannot be customized.
- ❖ **FR6.2: Limited Menu for Unauthenticated Users**
 - **Description:** The system shall display a simplified menu with login and exit options when no user is logged in.
 - **Details:** The `display_main_menu` function shows a login prompt with default credentials and options 1 (login) and 0 (exit) when `user_logged_in` is false.
 - **Constraints:** No other actions are available until login.
- ❖ **FR6.3: Sub-Menus for Modules**
 - **Description:** The system shall provide sub-menus for customer management, service management, billing, and reports, each with relevant options and a back-to-main-menu option.
 - **Details:** Each module (`customer_menu`, `serviceManagementMenu`, `billingManagementMenu`, `reportsMenu`) presents a numbered menu with module-specific options and a 0 to return to the main menu.
 - **Constraints:** Sub-menus are independent, with no cross-module navigation except via the main menu.
- ❖ **FR6.4: Input Validation for Navigation**
 - **Description:** The system shall prompt for user input and validate choices to ensure correct navigation.
 - **Details:** The `get_menu_choice` function validates numeric input using `scanf`. Invalid inputs (non-numeric or out-of-range) trigger an error message, and the input buffer is cleared using `clear_input_buffer`.
 - **Constraints:** Validation is basic, with no retry limit for invalid inputs.
 - **1.7 Data Persistence**
 - The system ensures data is retained between sessions using text files.
- ❖ **FR7.1: Data Directory Creation**
 - **Description:** The system shall create a data directory to store text files for customers, services, and invoices.
 - **Details:** The `create_data_directory` function uses platform-specific macros (`MKDIR`) to create the data directory if it does not exist, checking with `file_exists`.
 - **Constraints:** Directory creation is attempted only once at system startup. Failure is not retried.
- ❖ **FR7.2: Load Data from Files**
 - **Description:** The system shall load existing data from `customers.txt`, `services.txt`, and `invoices.txt` when performing relevant operations.
 - **Details:** Functions like `loadServices` and `loadSimpleInvoices` read text files into in-memory arrays at the start of their respective modules. Data is parsed using `sscanf` or `strtok` with pipe delimiters.
 - **Constraints:** File corruption or format errors may lead to partial or no data loading. No error recovery is implemented.

3.9 Non-Functional Requirements

Non-functional requirements define the system's quality attributes, such as performance, usability, reliability, and portability, ensuring it meets operational and user expectations.

3.10 Usability

- **NFR1.1: Intuitive Interface**
 - **Description:** The system shall provide a clear and intuitive command-line interface with formatted menus and prompts.
 - **Details:** Menus (display_main_menu, customer_menu, etc.) use ASCII art, color codes (e.g., \033[1m for bold), and consistent formatting for headers and tables. Prompts clearly indicate expected input (e.g., "Enter choice (1-4):").
 - **Rationale:** Ensures users can navigate and interact with the system without extensive training, suitable for a university project.
- **NFR1.2: Error Messages**
 - **Description:** The system shall display meaningful error messages for invalid inputs and guide users to correct them.
 - **Details:** Functions like get_menu_choice, addService, and search_customer_by_id display specific error messages (e.g., "Invalid choice!", "Customer ID not found") and pause for user acknowledgment.
 - **Rationale:** Enhances user experience by providing immediate feedback and preventing confusion.
- **NFR1.3: Consistent Output Formatting**
 - **Description:** The system shall use consistent formatting for output displays, such as tables for lists and receipts for invoices.
 - **Details:** Tables (e.g., in viewAllServices, viewAllInvoices) use fixed-width columns with headers and separators. Receipts in generateSimpleInvoice follow a professional layout with aligned text.
 - **Rationale:** Improves readability and maintains a professional appearance.
- **NFR1.4: Pause After Operations**
 - **Description:** The system shall pause after each operation and prompt the user to press Enter to continue.
 - **Details:** The pause_execution function uses a loop to wait for an Enter key press, ensuring users can review output before the screen clears.
 - **Rationale:** Prevents information from being cleared too quickly, allowing users to process results.

3.11 Reliability

- **NFR2.1: Input Error Handling**
 - **Description:** The system shall handle invalid inputs gracefully, preventing crashes and providing appropriate error messages.
 - **Details:** Functions like get_int_input, get_double_input, and get_string_input validate inputs using strtol, strtod, and fgets. Invalid inputs trigger error messages, and the input buffer is cleared.

- **Rationale:** Ensures system stability for a single-user environment.
- **NFR2.2: Data Integrity**
 - **Description:** The system shall ensure data integrity when reading from and writing to text files, avoiding corruption.
 - **Details:** File operations use `fopen`, `fgets`, and `fprintf` with error checking. Data is parsed carefully using `sscanf` or `strtok` to handle expected formats.
 - **Rationale:** Prevents loss or corruption of customer, service, or invoice data.
- **NFR2.3: Input Validation**
 - **Description:** The system shall validate critical inputs (e.g., customer IDs, service costs) to maintain consistent data.
 - **Details:** The `customerExists` function checks customer IDs, and `validate_year` ensures vehicle years are between 1900 and 2026. Costs are checked for positive values in `generateSimpleInvoice`.
 - **Rationale:** Reduces data inconsistencies, though some validations (e.g., phone number format) are not implemented.

3.12 Performance

- **NFR3.1: Data Loading Performance**
 - **Description:** The system shall load and display data (customers, services, invoices) within 2 seconds for up to 100 records.
 - **Details:** File reading in `loadServices`, `loadSimpleInvoices`, and `display_all_customers` processes small text files line by line, optimized for low record counts.
 - **Rationale:** Ensures quick response times for small-scale garage operations.
- **NFR3.2: Data Saving Performance**
 - **Description:** The system shall save data to text files within 1 second per operation.
 - **Details:** Functions like `saveSimpleInvoices` and `addService` write to files using `fprintf` in a single pass, handling up to 100 services or 50 invoices.
 - **Rationale:** Minimizes delays during data persistence operations.
- **NFR3.3: Memory Efficiency**
 - **Description:** The system shall support up to 100 services and 50 invoices in memory without performance degradation.
 - **Details:** Global arrays (services, invoices) are fixed at `MAX_SERVICES` (100) and `MAX_INVOICES` (50), using minimal memory for small datasets.
 - **Rationale:** Suitable for a university project with limited data volume.

3.13 Portability

- **NFR4.1: Cross-Platform Compatibility**
 - **Description:** The system shall be compatible with Windows and Unix-based systems (e.g., Linux, macOS) using platform-specific includes and macros.
 - **Details:** The code uses `#ifdef _WIN32` to include `<direct.h>` for Windows and `<sys/stat.h>` for Unix, with `MKDIR` macro for directory creation. Screen clearing uses `system("cls")` or `system("clear")`.

- **Rationale:** Ensures the system can run on different operating systems without modification.
- **NFR4.2: Standard C Libraries**
 - **Description:** The system shall use standard C libraries to ensure compatibility across compilers and operating systems.
 - **Details:** Only `<stdio.h>`, `<stdlib.h>`, `<string.h>`, and `<time.h>` are used, with platform-specific includes for directory operations.
 - **Rationale:** Simplifies compilation and deployment for a university project.

3.14 Security

- **NFR5.1: Restricted Access**
 - **Description:** The system shall restrict access to authenticated users only, using a simple username/password mechanism.
 - **Details:** The `is_user_logged_in` function checks the `user_logged_in` flag before allowing access to management modules.
 - **Rationale:** Provides basic security for a single-user system.
- **NFR5.2: No Plain-Text Sensitive Data**
 - **Description:** The system shall not store sensitive data (e.g., passwords) in plain text, except for hardcoded default credentials.
 - **Details:** Passwords are hardcoded in the code (`DEFAULT_USERNAME`, `DEFAULT_PASSWORD`) but not stored in files. Other data (e.g., customer details) is stored in plain text.
 - **Rationale:** Minimal security is acceptable for a university project with no real-world sensitive data.

3.15 Maintainability

- **NFR6.1: Single-File Structure**
 - **Description:** The system shall be implemented in a single C file with clear section comments and function documentation.
 - **Details:** The code is organized with section headers (e.g., "AUTHENTICATION SYSTEM", "CUSTOMER MANAGEMENT SYSTEM") and function comments explaining purpose and parameters.
 - **Rationale:** Simplifies submission and review for a university project.
- **NFR6.2: Modular Functions**
 - **Description:** The system shall use modular functions with specific responsibilities to facilitate updates or extensions.
 - **Details:** Functions like `add_customer`, `addService`, and `generateSimpleInvoice` are self-contained, handling specific tasks with minimal dependencies.
 - **Rationale:** Makes it easier to modify or extend individual features.
- **NFR6.3: Configurable Constants**
 - **Description:** The system shall define constants and types at the beginning for easy configuration.

- **Details:** Constants like MAX_SERVICES, MAX_INVOICES, and MAX_NAME_LENGTH are defined at the top, along with typedef structures (customer_t, Service, Invoice).
- **Rationale:** Allows quick adjustments to limits or formats without changing core logic.

3.16 Scalability

- **NFR7.1: Data Limits**
 - **Description:** The system shall support up to 100 services and 50 invoices, with error messages when limits are reached.
 - **Details:** The addService and generateSimpleInvoice functions check MAX_SERVICES and MAX_INVOICES before adding new records, displaying errors if exceeded.
 - **Rationale:** Sufficient for small garage operations, aligning with project scope.
- **NFR7.2: Text File Storage**
 - **Description:** The system shall use simple text files for data storage, suitable for small-scale operations.
 - **Details:** Data is stored in customers.txt, services.txt, and invoices.txt using pipe-delimited text, readable by external tools.
 - **Rationale:** Simplifies implementation and meets the needs of a small-scale system.

3.17 Compatibility

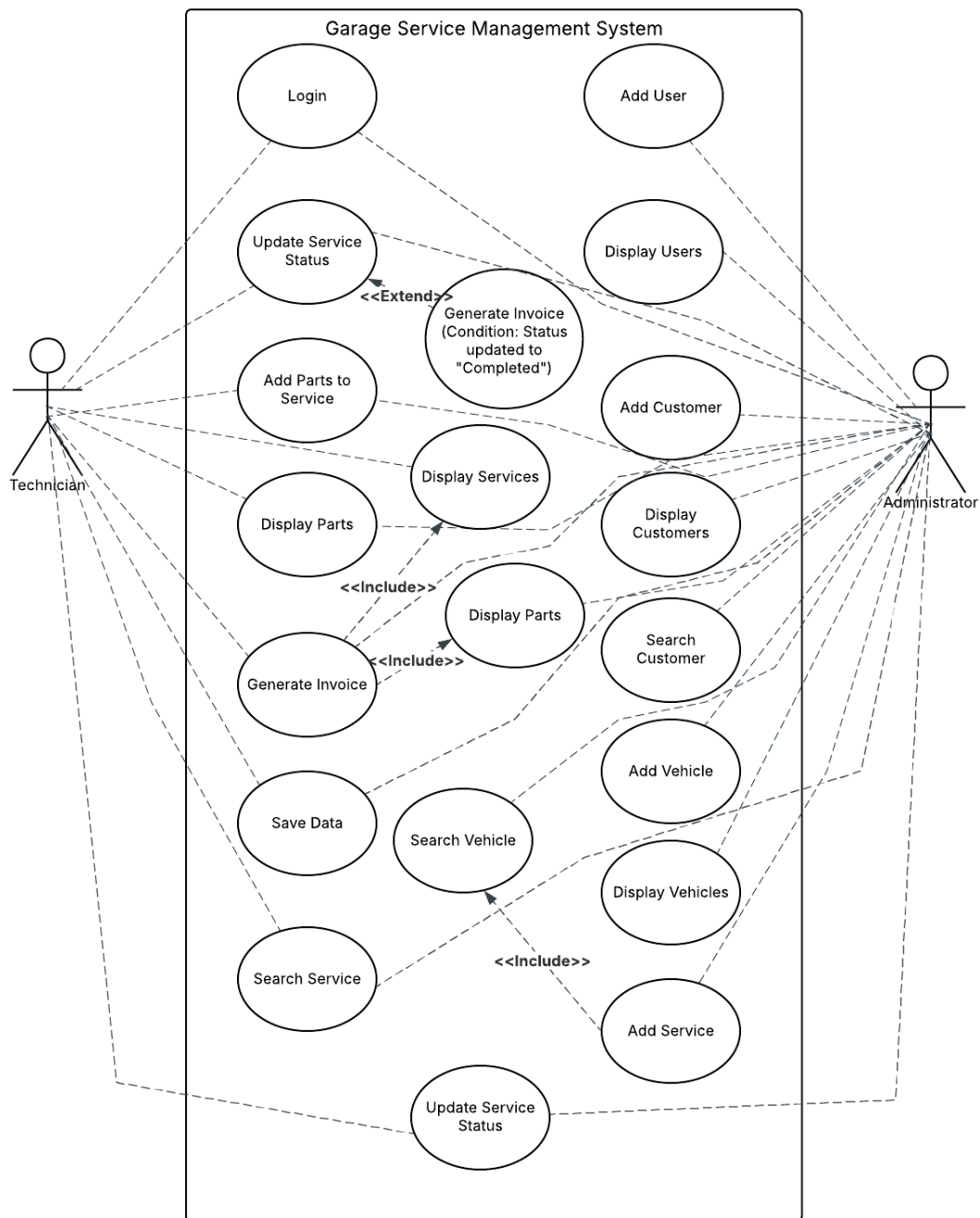
- **NFR8.1: Text File Format**
 - **Description:** The system shall store data in pipe-delimited text files for compatibility with external tools (e.g., spreadsheets).
 - **Details:** Files use a consistent | delimiter, with fields in a predefined order, allowing import into tools like Excel or CSV parsers.
 - **Rationale:** Enables data analysis or backup outside the system.
- **NFR8.2: Standard Date Format**
 - **Description:** The system shall use the YYYY-MM-DD date format for consistency and compatibility.
 - **Details:** Functions like getCurrentDate and getCurrentReportDate use strftime to format dates as "YYYY-MM-DD".
 - **Rationale:** Ensures dates are universally readable and sortable.

3.18 Constraints

- **NFR9.1: C Language Implementation**
 - **Description:** The system shall be implemented in C without external dependencies beyond standard libraries and platform-specific includes.
 - **Details:** The code uses only standard C libraries and platform-specific headers for directory operations, ensuring portability.
 - **Rationale:** Meets university project requirements for simplicity and portability.

- **NFR9.2: Single-User Environment**
 - **Description:** The system shall assume a single-user environment with no concurrent access to data files.
 - **Details:** File operations (fopen, fwrite) do not implement locking or concurrency controls, as only one user is expected.
 - **Rationale:** Simplifies implementation for a small-scale academic project.
- **NFR9.3: Simple Authentication**
 - **Description:** The system shall use a simple authentication mechanism with hardcoded credentials.
 - **Details:** The validate_credentials function checks against "admin"/"admin" without supporting user management or encryption.
 - **Rationale:** Sufficient for a proof-of-concept system in an academic setting.

4. Use Case Diagram



Use case description:

The following table provides detailed descriptions of each use case in the system.

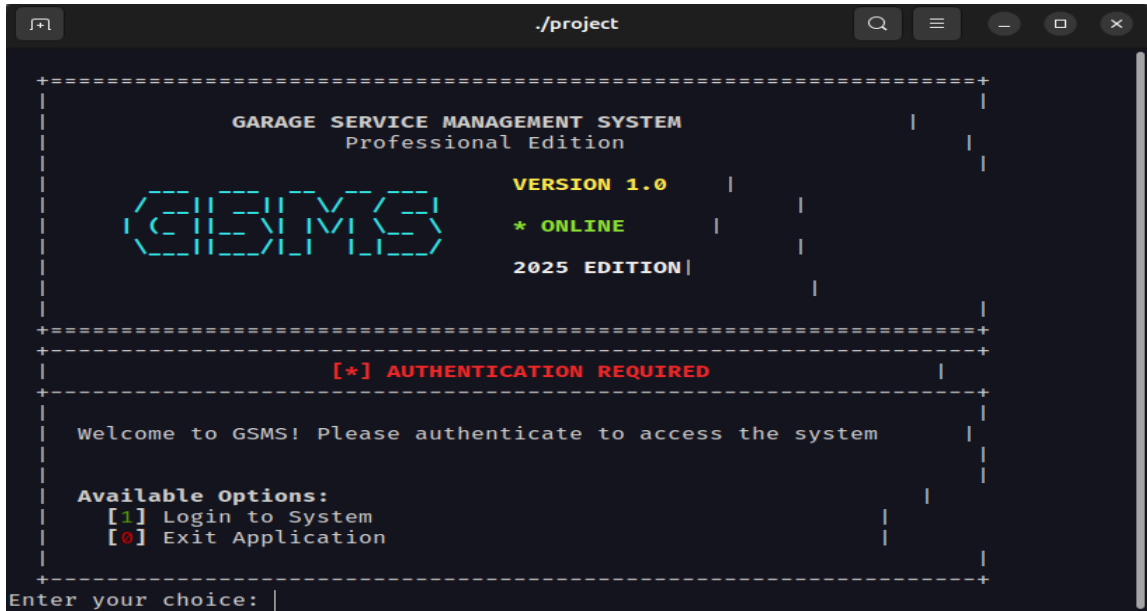
Use Case	Description	Actor
----------	-------------	-------

User Authentication	Allows authorized staff to log in and out of the system using predefined credentials to secure access.	Garage Staff
Customer & Vehicle Registration	Enables staff to add new customers and their associated vehicle details (plate, make, model, etc.) into the system.	Garage Staff, Customer
Customer Management	Allows staff to view a list of all registered customers and search for a specific customer using their unique ID.	Garage Staff
Service Registration	Records a new service request for a customer, including the service description, cost, and initial "Pending" status.	Garage Staff
Service Management	Enables staff to view all services and update the status of a service (e.g., from "Pending" to "In Progress" or "Completed").	Garage Staff
Invoice Generation	Generates and displays a printable invoice for a completed service, detailing the customer, service, and total amount.	Garage Staff, Customer
Billing Management	Allows staff to view a complete history of all invoices that have been generated by the system.	Garage Staff
Daily Revenue Reporting	Generates a financial report that summarizes the total revenue earned from all invoices created on the current day.	Garage Staff

5. User Manual

1. Start

This is the main menu of the Garage Service Management System. It provides options to Login as admin or Exit. Users select a choice by entering the corresponding number.



```
./project
=====
GARAGE SERVICE MANAGEMENT SYSTEM
Professional Edition

  GSMS  VERSION 1.0
        * ONLINE
        2025 EDITION

=====
[*] AUTHENTICATION REQUIRED
=====

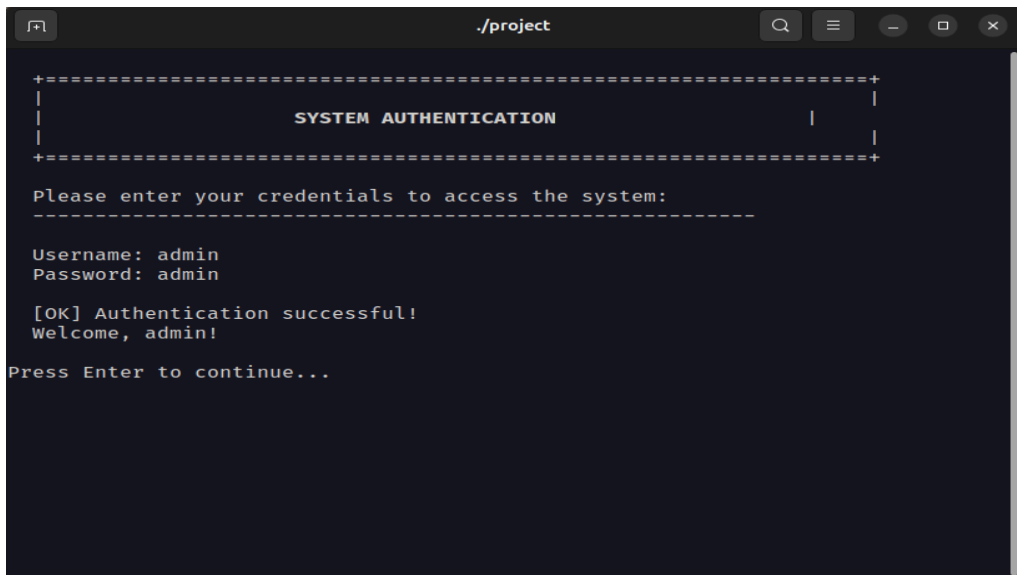
Welcome to GSMS! Please authenticate to access the system

Available Options:
[1] Login to System
[0] Exit Application

=====
Enter your choice: |
```

2. Login

This option allows the admin to login using username and password. Upon successful login, a confirmation message ("[OK] Authentication successful!") is displayed, and the Main Menu is shown.



```
./project
=====
SYSTEM AUTHENTICATION
=====

Please enter your credentials to access the system:
-----

Username: admin
Password: admin

[OK] Authentication successful!
Welcome, admin!

Press Enter to continue...
```

3. Main Menu

After login, the main menu is displayed with options for Customer & Vehicle Management, Service Management, Billing & Invoices, Reports, Logout, or Exit.

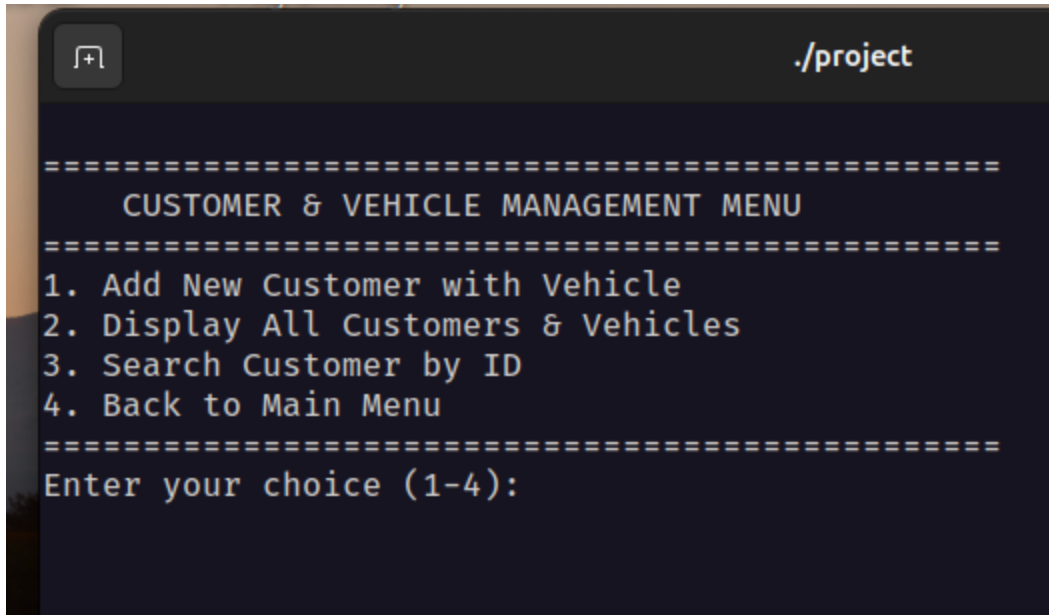
```
./project
```

```
+=====+  
|                                     |  
|      \o/  \o/  \o/  \o/  \o/    * SYSTEM ACTIVE |  
|   \o/  \o/  \o/  \o/  \o/       MANAGEMENT     |  
|   \o/  \o/  \o/  \o/  \o/       SYSTEM          |  
|                                     |  
+=====+  
  
>> Welcome back, admin  
-----  
  
[*] SYSTEM MODULES:  
  
[1] >> Customer & Vehicle Management  
[2] >> Service Management  
[3] >> Billing & Invoices  
[4] >> Reports  
  
[*] SYSTEM OPTIONS:  
  
[5] >> Logout  
[0] >> Exit  
  
Enter your choice: |
```

5.2 Feature Description for Customer & Vehicle Management

- **Customer & Vehicle Menu**

This submenu allows admin to add a new customer with vehicle, display all customers and vehicles, search by ID, or return to the Main Menu.



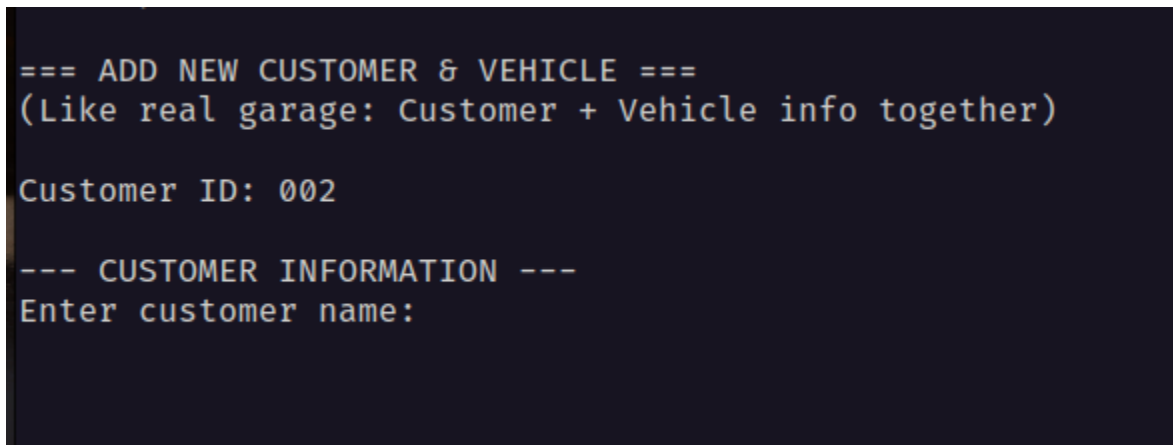
```

./project

=====
      CUSTOMER & VEHICLE MANAGEMENT MENU
=====
1. Add New Customer with Vehicle
2. Display All Customers & Vehicles
3. Search Customer by ID
4. Back to Main Menu
=====
Enter your choice (1-4):

```

-
- **Add New Customer with Vehicle**
This option allows admin to enter customer details (name, phone, address) and vehicle details (plate, make, model, year, color). Upon success, a confirmation message (“[OK] Customer & Vehicle added successfully!”) is displayed.



```

=== ADD NEW CUSTOMER & VEHICLE ===
(Like real garage: Customer + Vehicle info together)

Customer ID: 002

--- CUSTOMER INFORMATION ---
Enter customer name:

```

- **Display All Customers & Vehicles**
This option displays a table of all customers with their vehicle information, including ID, Name, Phone, Plate, Make, and Model.

```

Enter your choice (1-4): 2

=== ALL CUSTOMERS & THEIR VEHICLES ===
ID      Name          Phone          Plate          Make          Model
-----
001     Hridoy          01711589635    ADIE-333        BMW           G-wagon
-----
Total customers: 1
(Each customer registered with their vehicle)

Press Enter to continue...|

```

- **Search Customer by ID**

This option allows admin to search for a customer by ID. If found, customer and vehicle details are displayed; otherwise, an error message (“[X] Customer with ID not found.”) is shown.

```

=====
Enter your choice (1-4): 3
Enter customer ID to search: 001

=== SEARCH CUSTOMER BY ID ===
[OK] Customer Found!
=====
CUSTOMER INFORMATION:
ID: 001
Name: Hridoy
Phone: 01711589635
Address: Dhaka bangladesh

VEHICLE INFORMATION:
License Plate: ADIE-333
Vehicle: 2021 BMW G-wagon
Color:
=====

Press Enter to continue...

```

5.3 Feature Description for Service Management

- **Service Management Menu**

This submenu allows admin to add a new service, view all services, update service status, or return to the Main Menu.

```

./project

=== Service Management ===
1. Add New Service
2. View All Services
3. Update Service Status
0. Back to Main Menu

Enter your choice: |

```

- **Add New Service**

Admin enters Customer ID, Service Name, and Cost. Upon success, a confirmation message (“[OK] Service added successfully!”) is displayed.

```

Enter your choice: 1

Enter Customer ID: 001

=== SEARCH CUSTOMER BY ID ===
[OK] Customer Found!
=====
CUSTOMER INFORMATION:
ID: 001
Name: Hridoy
Phone: 01711589635
Address: Dhaka bangladesh

VEHICLE INFORMATION:
License Plate: ADIE-333
Vehicle: 2021 BMW G-wagon
Color:
=====
Enter Service Name: |

```

- **View All Services**

This option displays all services in a table with Service ID, Customer ID, Service Name, Date, Status, and Cost.

```

Enter your choice: 2

=== All Services ===
ID | Customer ID | Service Name | Date | Status | Cost
-----+-----+-----+-----+-----+-----
001 | 1 | test | 2025-08-21 | Pending | $100.00
002 | 1 | ss | 2025-08-21 | Pending | $44.00

Press Enter to continue...|

```

- **Update Service Status**

Admin enters Service ID and selects a new status (Pending, In Progress, Completed). Upon success, a confirmation message (“[OK] Service status updated successfully!”) is displayed.

```

Enter your choice: 3

Enter Service ID to update (like 001): 001

Current Status: Pending

Select New Status:
1. Pending
2. In Progress
3. Completed
Enter choice (1-3): 3

```

5.4 Feature Description for Billing & Invoices

- **Billing & Invoices Menu**

This submenu allows admin to generate an invoice, view all invoices, or return to the Main Menu.

```

=== Simple Billing System ===
1. Generate Invoice
2. View All Invoices
0. Back to Main Menu

Enter your choice: |

```

- **Generate Invoice**

Admin selects a completed service, and an invoice is generated with details like Invoice

ID, Customer Name, Service Name, and Total Amount. A receipt can be printed if chosen, with a confirmation message (“[OK] Invoice created successfully!”).

```

Enter your choice: 1

=== Generate Simple Invoice ===

--- Completed Services ---
No. | Service ID | Customer ID | Service Name | Cost
-----+-----+-----+-----+-----
1   | 001       | 1         | test        | $100.00

Select service number (1-1): 001

[OK] Invoice created successfully!
Invoice ID: INV001
Total Amount: $100.00

Print receipt? (y/n): |

```

- **View All Invoices**

This option displays all invoices in a table with Invoice ID, Customer Name, Service Name, Amount, and Status.

```

Enter your choice: 2

=== All Invoices ===
Invoice ID | Customer Name | Service Name | Amount | Status
-----+-----+-----+-----+-----
INV001    | Hridoy       | test        | $100.00 | Paid

Press Enter to continue...

```

5.5 Feature Description for Reports

- **Reports Menu**

This submenu allows admin to view the Daily Revenue Report or return to the Main Menu.

```

j+| ./.project

=== Reports Menu ===
1. Daily Revenue Report
0. Back to Main Menu

Enter your choice:

```

- **Daily Revenue Report**

This option displays invoices for the current date, total revenue, and details in a table format with Invoice ID, Customer Name, Service Name, Amount, and Status.

```

=== Daily Revenue Report ===
Date: 2025-08-21
-----

Invoice Details for Today:
Invoice ID | Customer Name      | Service Name  | Amount  | Status
-----+-----+-----+-----+-----
INV001    | Hridoy             | test          | $100.00 | Paid

-----

Total Invoices Today: 1
Total Revenue Today: $100.00
-----

Press Enter to continue...|

```

- **Logout**

This option allows admin to logout. After logout, the initial menu is displayed with the message (“[OK] Logout successful!”).

```

+=====+
|                                     |
|                               SYSTEM LOGOUT                               |
|                                     |
+=====+

Logging out user: admin
-----

[OK] Logout successful!
Thank you for using the system.

Press Enter to continue...|

```

- **Exit**

This option allows the admin to exit the application. Upon selecting this option, the program will close, ending the current session.

```

+-----+
|               [*] AUTHENTICATION REQUIRED               |
+-----+
| Welcome to GSMS! Please authenticate to access the system |
|
| Available Options:                                     |
|   [1] Login to System                                   |
|   [0] Exit Application                                   |
|
+-----+

Enter your choice: 0

>> Goodbye! Thank you for using GSMS.

[*] Cleaning up system resources...
[OK] System cleanup completed.

Thank you for using GSMS!
Goodbye!

```

6. Project Plan

Timeline:

Requirement Analysis	1 week
Design & Diagram	1 week
Coding	3 weeks
Testing & Feedback	2 weeks
Deployment & Training	2 weeks

Deliverables :

1. User Authentication Module:	A secure login and logout system to restrict system access to authorized staff members only.
2. Customer & Vehicle Records Module:	A complete module for adding, searching by ID, and displaying customer profiles along with their vehicle details.
3. Service Tracking Module:	Functionality to record new garage services and dynamically track their real-time status (Pending, In Progress, Completed).
4. Automated Invoicing System:	A system to automatically generate invoices based on completed services, with an option to display a formatted, printable receipt.
5. Data Persistence Engine:	A file-based storage engine that saves all system data to .txt files, ensuring information is retained between sessions.
6. Cross-Platform Compatibility:	Source code designed to compile and run correctly on both Windows and Unix-like operating systems (e.g., Linux, macOS).
7. Fully Commented Source Code:	The complete, single-file C program with clear comments explaining functions and logic, ready for academic submission and review.
8. Complete Project Documentation:	All necessary documentation, including the project plan, use case descriptions, and a user guide for system operation.

7. Resources (Required Personnel, Tools, and Materials)

Personnel:

❖ Project Manager (Oversees timeline and deliverables)
❖ System Analyst (Defines project requirements and scope)
❖ Software Architect (Designs the system structure, modules, and data flow)
❖ Lead C Programmer (Writes and implements the core application logic)
❖ Interface Designer (CLI) (Designs the user-friendly menu and console interaction)
❖ Data Manager (Manages the file-based storage and data integrity)
❖ Quality Assurance (QA) Engineer (Tests the system for bugs and ensures it meets requirements)
❖ Technical Writer (Prepares the project documentation and user manual)

Tools

* Programming Language: C
* Compiler: GCC (MinGW for Windows), Clang
* Development Environment: Visual Studio Code, Code::Blocks
* Database: File-based storage system using plain text (.txt) files
* Version Control: Git, GitHub
* Debugging: GDB (GNU Debugger), Manual code walkthroughs, and printf statements

Materials:

❖ * Project Requirements Document
--

❖ * Application Flowchart
❖ * Data Structure & File Format Design
❖ * Test Plan & Scenarios
❖ * C Source Code File
❖ * Final Project Report & Documentation

8. Budget

Cost Estimates:

Category	Cost Estimate (Notional Units)
Software Tools	0 (Open-source)
Development Time	12,000
Testing & Debugging	5,000
Documentation & Reporting	3,000
Hardware	0 (Standard PC used)
Miscellaneous (Planning, Research)	1,500
Total Estimated Cost	21,500

9. Testing and Analysis for the whole system

System testing is a critical phase in the software development lifecycle where a complete, integrated system is tested to verify that it meets the specified requirements. This level of testing is performed after integration testing and before acceptance testing. The main goal of system testing is to evaluate the system's compliance with the specified requirements. It involves testing the system as a whole in a fully integrated environment to ensure that it functions correctly and reliably.

9.1 Testing Analysis for whole system

The Garage Service Management System, developed in C, has undergone comprehensive system testing to ensure it meets all functional and non-functional requirements.

Our Objectives Were:

- **Functional Validation:** Confirm all features and functionalities work as intended.
- **Non-Functional Evaluation:** Assess performance, usability, and reliability.
- **Defect Identification and Resolution:** Detect and resolve issues before deployment.
- **User Experience Assurance:** Ensure the system is intuitive and user-friendly.

Testing Outcomes:**1. Functional Testing:**

For Admin:

- Successfully logged in and out.
- Added customers with vehicle details.
- Viewed and searched customers by ID.
- Added and updated services.
- Generated and viewed invoices.
- Generated daily revenue reports.

2. Non-Functional Testing:

- **Performance:** The system performed well under normal load conditions, maintaining responsiveness and stability.
- **Security:** Successfully validated credentials for access control.
- **Usability:** The console interface is clear and easy to navigate.
- **Cross-Platform Compatibility:** Successfully tested on Windows and Unix-like systems.

3. Regression Testing:

- Verified that code changes did not impact existing functionalities.
- Ensured that bug fixes did not introduce new issues.

4. End-to-End Testing:

- Confirmed seamless workflow from login to adding customers, services, generating invoices, and viewing reports.

Defects Identified and Resolved:

- **Defect Logging:** All identified defects were logged with detailed descriptions and reproduction steps.
- **Defect Tracking:** Utilized a defect tracking tool to monitor the status and resolution of each defect.
- **Defect Resolution:** Prioritized and addressed defects based on their severity and impact on the system.

Documentation and Reporting:

- Maintained comprehensive documentation, including test plans, test cases, test results, and defect logs.
- Generated detailed test reports summarizing the testing activities, outcomes, and identified issues.

The Garage Service Management System has successfully passed through rigorous system testing, demonstrating its readiness for deployment. All functionalities have been validated, non-functional requirements have been met, and identified defects have been resolved. The system is now verified to be reliable, secure, and user-friendly, ensuring a smooth and efficient experience for all stakeholders. The next step is deployment, where continuous monitoring and minor adjustments will ensure optimal performance in the production environment.

10. Conclusion

This project created a Garage Service Management System that improves the service and billing process between the admin and customers. By making the system easy to use and secure, it saves time, reduces errors, and ensures satisfaction for both admin and customers.

11. Future Work

- Implementing an **inventory management** system for garage **parts**.
- Adding **advanced analytics** and visual **reports** for administrators to track business performance.
- Developing a **graphical user interface (GUI)** to enhance usability and make operations more efficient.

11. References

Books:

- C Programming Books & Tutorials
- GARAGE Service Management System Research Papers
- Online Resources for Software Development

Website:

Websites:

- Stack Overflow (<https://stackoverflow.com>)
 - GeeksforGeeks (<https://www.geeksforgeeks.org/c-programming-language/>)
 - TutorialsPoint (<https://www.tutorialspoint.com/cprogramming/index.htm>)
 - CProgramming.com (<https://www.cprogramming.com>)
 - GitHub (<https://github.com>)
 - YouTube (<https://youtube.com>)
 - AI Assistants (e.g., claude 4.0 opus, ChatGPT)
-

12. Supplementary Information

❖ Sample Data: Example customers.txt, services.txt, and invoices.txt files used for testing and demonstration purposes.
❖ Compilation & Execution Guide: Step-by-step instructions for compiling the C source code with a GCC compiler and running the executable.
❖ User Manual: A guide for garage staff detailing how to perform all major operations, from login to generating reports.
❖ Test Case Log: Documentation of test cases for each module, including inputs, expected outputs, and actual results to validate functionality.
❖ Error Handling Overview: A brief guide on how the system manages common errors, such as invalid user input or file access issues.
❖ Security Overview: Details on the basic hardcoded user authentication implemented to restrict access to the system.
❖ Development Log: A log outlining the development progress, features implemented, and significant bugs fixed during the project timeline.